

RETAIL DEMAND FORECASTING

STUDENT 1 — PROPER CONTRIBUTION DOCUMENTATION

Data Ingestion & HDFS Storage

Project	Retail Demand Forecasting
Student Role	Student 1 — Data Ingestion + HDFS
Dataset	retail_demand.csv
Local Dataset Path	C:\big_data_lab\Retail_Demand_Forecasting\data\retail_demand.csv
HDFS Raw Path	/data/retail/raw/retail_demand.csv
Records	76,000
Columns	16
Uploaded File Size	6.1 MB in HDFS
Hadoop	Apache Hadoop 3.3.6

This document focuses specifically on Student 1's individual contribution: preparing the local Hadoop environment, creating the HDFS project structure, uploading the retail demand dataset, verifying storage, and preparing the raw data foundation for Student 2.

1. Student 1 Contribution — Overview

Student 1 is responsible for the **Data Ingestion + HDFS Storage** layer of the Retail Demand Forecasting project. The purpose of this stage is to take the project CSV dataset from the local file system and make it available as distributed data in HDFS.

- Prepare and verify the local Hadoop/HDFS environment.
- Configure HDFS NameNode and DataNode storage.
- Create the project-specific HDFS directory structure.
- Upload the retail demand CSV into the HDFS raw layer.
- Verify that the file exists in HDFS.
- Check HDFS storage usage and DataNode health.
- Verify that the stored file contains the expected header and records.

The output of Student 1 is the **raw HDFS data layer** that Student 2 can use as the basis for Hive data modeling.

2. Project Objective

The overall project is Retail Demand Forecasting, with the objective of analyzing historical demand and demand patterns. Student 1 establishes the distributed storage foundation required for the later Hive and Spark stages.

The Student 1 stage answers the first infrastructure question: **How do we take the project dataset from a local CSV file and make it available in HDFS for Big Data processing?**

3. Dataset Used

Property	Details
Dataset	retail_demand.csv
Rows	76,000
Columns	16
Target variable	Demand
Local source	C:\big_data_lab\Retail_Demand_Forecasting\data\retail_demand.csv
HDFS raw destination	/data/retail/raw/retail_demand.csv

The dataset contains time information, store/product identifiers, category and region, inventory and sales measures, pricing/discount fields, weather and promotion information, seasonality, epidemic information, and the demand target.

4. Dataset Schema

Column	Role
Date	Date/time field
Store ID	Store identifier
Product ID	Product identifier
Category	Product category
Region	Geographic region
Inventory Level	Inventory measure
Units Sold	Sales measure
Units Ordered	Order measure
Price	Product price
Discount	Discount value
Weather Condition	External weather factor

Column	Role
Promotion	Promotion indicator
Competitor Pricing	Competitor price
Seasonality	Seasonal factor
Epidemic	External event indicator
Demand	Target demand value

The CSV was inspected before ingestion and contained 76,000 non-null records across all 16 columns.

5. Local Hadoop Environment

Student 1 used a locally installed Hadoop environment rather than Docker.

Component	Configuration
Hadoop	Apache Hadoop 3.3.6
Java	JDK 17
HADOOP_HOME	C:\big_data_lab\hadoop\hadoop-3.3.6
JAVA_HOME	C:\Program Files\Java\jdk-17
HDFS NameNode address	hdfs://localhost:9000
NameNode storage	C:\big_data_lab\hadoop_data\namenode
DataNode storage	C:\big_data_lab\hadoop_data\datanode
Replication factor	1

The local Hadoop configuration used **fs.defaultFS = hdfs://localhost:9000**. The HDFS replication factor was configured as 1 for the local development environment.

6. HDFS Configuration

core-site.xml configured the default HDFS endpoint:

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

hdfs-site.xml configured replication and local NameNode/DataNode storage:

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///C:/big_data_lab/hadoop_data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///C:/big_data_lab/hadoop_data/datanode</value>
  </property>
</configuration>
```

7. Hadoop Startup and Troubleshooting

During setup, the Hadoop daemons did not initially start correctly. The startup was diagnosed using **jps**. The first check showed only the Jps process rather than the Hadoop daemons.

- DataNode startup was attempted using start-dfs.cmd.
- The NameNode initially failed because the NameNode storage directory had an access-permission problem.

- The permission problem was resolved by granting the Hadoop data directory full access to the local user.

```
icacls "C:\big_data_lab\hadoop_data" /grant aisha:(OI)(CI)F /T
```

The command processed the Hadoop data directory successfully. The NameNode was then started directly using:

```
%HADOOP_HOME%\bin\hdfs.cmd namenode
```

The NameNode log confirmed Hadoop 3.3.6, Java 17, and **fs.defaultFS = hdfs://localhost:9000**. The NameNode web service started on port 9870.

8. DataNode Verification

The DataNode was started through the Hadoop DFS startup script. HDFS communication was verified through **hdfs dfs** commands and later through **hdfs dfsadmin -report**.

The DataNode report showed one live DataNode, no under-replicated blocks, no corrupt replicas, and no missing blocks.

HDFS Health Item	Observed Result
Live DataNodes	1
Under-replicated blocks	0
Corrupt replicas	0
Missing blocks	0
DFS remaining	Approximately 106.78 GB
Configured capacity	Approximately 474.72 GB

This confirmed that the local HDFS storage layer was operational before final dataset ingestion.

9. Creating the HDFS Project Structure

The project uses separate directories for raw input, processed data, and result outputs:

```
/data/retail/
■■■ raw/
■■■ processed/
■■■ results/
```

The directories were created using:

```
hdfs dfs -mkdir -p /data/retail/raw
hdfs dfs -mkdir -p /data/retail/processed
hdfs dfs -mkdir -p /data/retail/results
```

Directory	Purpose
/data/retail/raw	Original uploaded retail CSV
/data/retail/processed	Future cleaned/transformed data
/data/retail/results	Future analytics/result outputs

10. Uploading the Dataset into HDFS

The dataset was uploaded from the local project directory using:

```
hdfs dfs -put -f "C:\big_data_lab\Retail_Demand_Forecasting\data\retail_demand.csv" /data/retail/raw/
```

The first upload attempt failed because the DataNode was not running at that moment. The DataStreamer reported a connection refusal to port 9866. The DataNode was started, its health was checked, and the upload was retried successfully.

This troubleshooting step is important because it demonstrates that the ingestion process depends on an active DataNode.

11. HDFS File Verification

After the successful upload, the raw directory was checked using:

```
hdfs dfs -ls -h /data/retail/raw
```

File	Size	Status
/data/retail/raw/retail_demand.csv	6.1 M	Successfully stored

The file was also checked using HDFS disk-usage reporting:

```
hdfs dfs -du -h /data/retail/raw
```

The output reported approximately **6.1 MB** for the file.

12. Verifying the Actual File Content

The first portion of the HDFS file was inspected using:

```
hdfs dfs -head /data/retail/raw/retail_demand.csv
```

The output showed the expected CSV header:

```
Date,Store ID,Product ID,Category,Region,Inventory Level,Units Sold,Units Ordered,Price,Discount,Weather Condition,Promotion,Comments
```

The first records matched the original dataset, including examples such as S001/P0001 Electronics North and S001/P0002 Clothing North. This confirms that the file was transferred correctly rather than merely existing as an HDFS object.

13. Complete Student 1 Data-Ingestion Flow

```
Local Project Dataset
C:\big_data_lab\Retail_Demand_Forecasting\data\retail_demand.csv
|
| hdfs dfs -put
v
HDFS NameNode / DataNode
|
v
/data/retail/raw/retail_demand.csv
|
+--> hdfs dfs -ls
|
+--> hdfs dfs -du
|
+--> hdfs dfs -head
|
v
Verified Raw HDFS Data Layer
|
v
Student 2 – Hive Data Modeling + SQL Analytics
|
v
Student 3 – Spark / PySpark
```

This shows exactly how Student 1 feeds Student 2: Student 1 does not perform Hive modeling; Student 1's output is the verified raw HDFS data that Student 2 can consume.

14. Handoff from Student 1 to Student 2

The Student 1 handoff consists of the following agreed artifacts:

Handoff Item	Value
Dataset	retail_demand.csv
Record count	76,000
Column count	16
HDFS raw path	/data/retail/raw/retail_demand.csv
Raw directory	/data/retail/raw

Handoff Item	Value
Future processed directory	/data/retail/processed
Future results directory	/data/retail/results
Target field	Demand

If Student 2 is working on a separate cloud/Docker environment, the physical HDFS instance does not need to be accessed directly. The same master CSV and schema can be transferred/shared, and Student 2 can recreate the raw HDFS location in that environment before creating the Hive table.

15. Formal Contribution Statement

The following paragraph can be used directly in the final project report:

Student 1 was responsible for the Data Ingestion and HDFS Storage layer of the Retail Demand Forecasting project. The project dataset, `retail_demand.csv`, containing 76,000 records and 16 columns, was prepared in the local project directory and uploaded to a locally installed Apache Hadoop 3.3.6 HDFS environment. The Hadoop configuration was established using a local NameNode and DataNode, with HDFS configured at `hdfs://localhost:9000` and a replication factor of 1. Project-specific HDFS directories for raw, processed, and result data were created. The raw CSV was uploaded to `/data/retail/raw/retail_demand.csv` and verified using HDFS listing, disk-usage, and file-content commands. DataNode health and HDFS storage status were also checked. The verified raw HDFS dataset was then provided as the storage-layer foundation for Student 2's Hive data modeling and SQL analytics stage.

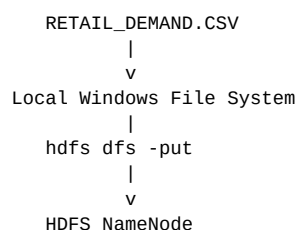
16. Issues Encountered and Resolutions

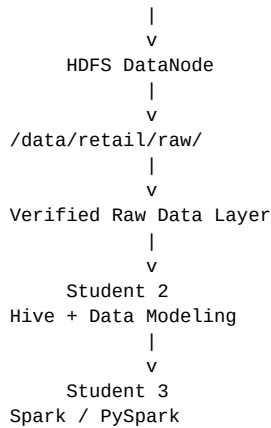
Issue	Resolution
NameNode/DataNode not initially visible in jps	Started and diagnosed Hadoop daemons.
NameNode permission error on VERSION file	Granted permissions on <code>C:\big_data_lab\hadoop_data</code> using <code>icacls</code> .
DataNode connection refused during first upload	Started the DataNode and verified it with <code>hdfs dfsadmin -report</code> , then retried upload.
Windows 'more' command unavailable for piping HDFS output	Used <code>hdfs dfs -head</code> directly to inspect the beginning of the file.
<code>hdfs -head</code> initially used incorrectly as a pipe	Corrected to <code>hdfs dfs -head <HDFS-file-path></code> .

17. Evidence Produced by Student 1

- Hadoop version output confirming Apache Hadoop 3.3.6.
- HDFS configuration files: `core-site.xml` and `hdfs-site.xml`.
- HDFS project directory structure.
- Successful CSV upload to `/data/retail/raw`.
- HDFS file listing showing the 6.1 MB file.
- HDFS disk-usage output.
- HDFS file-content/header verification.
- DataNode/HDFS health report.
- Final raw-data HDFS path for Student 2.

18. Student 1 Architecture





Student 1 is therefore the first technical stage in the distributed analytics pipeline.

19. Student 1 Completion Checklist

Task	Status
Dataset identified and inspected	✓
Local Hadoop 3.3.6 verified	✓
HDFS configuration completed	✓
NameNode configured	✓
DataNode configured	✓
HDFS connectivity verified	✓
Raw/processed/results directories created	✓
retail_demand.csv uploaded	✓
HDFS file listing verified	✓
Storage usage verified	✓
File contents/header verified	✓
DataNode health verified	✓
Raw dataset prepared for Student 2	✓

20. Viva / Presentation Explanation

What was your role?

My role was Data Ingestion and HDFS Storage. I prepared the local Hadoop environment, created the HDFS directory structure, uploaded the retail demand CSV into the raw HDFS directory, and verified the file and HDFS health.

Why did you use HDFS?

HDFS provides distributed storage for Big Data workloads. In this project it acts as the storage layer from which Hive and later Spark can process the retail demand data.

What was your output?

My main output was the verified raw dataset stored at /data/retail/raw/retail_demand.csv, together with the HDFS directory structure and storage verification evidence.

How did Student 2 use your work?

Student 2 used the same project dataset and HDFS storage concept as the input to the Hive stage. In a separate environment, Student 2 can reproduce the raw HDFS path and then create the Hive external table over it.

What happened when the first upload failed?

The DataNode was not running, so the DataStreamer could not connect to port 9866. I started the DataNode, checked the HDFS report, and then repeated the upload successfully.

21. Conclusion

Student 1 successfully established the raw distributed storage foundation for the Retail Demand Forecasting project. The 76,000-record, 16-column CSV was transferred from the local project directory into HDFS, stored under the raw-data layer, and verified through file listing, disk usage, file-content inspection, and DataNode health checks.

The key hand-off from Student 1 is the verified raw dataset and its defined HDFS location. Student 2 extends this foundation by creating the Hive schema and SQL analytics layer, after which Student 3 performs Spark/PySpark processing.